Live Cricket Score Management System

A Project Report

Submitted in partial fulfillment of the

Requirements for the award of the Degree of

BACHELOR OF SCIENCE (COMPUTER SCIENCE)

By

Tejas Vijay Modke

Class: T.Y.B.Sc. (Computer Science) - TYCS

2026012040(510515147)

Under the esteemed guidance of

Asst. Prof. Saba Ansari

Head of Department



DEPARTMENT OF COMPUTER TECHNOLOGY

J. K. COLLEGE OF SCIENCE AND COMMERCE (AUTONOMOUS)

Affiliated to University of Mumbai

NAVI MUMBAI, 400701

MAHARASHTRA

2026-27

ABSTRACT

Cricket is played at every level, from school and society tournaments to academies and clubs, yet scoring in many of these matches is still done manually on paper scorebooks. Manual scoring is slow, prone to errors in runs, overs and extras, and makes it difficult to preserve and analyse match records. Live Cricket Score Management System is a web application designed to provide a simple and reliable platform for managing teams, players, matches and live ball-by-ball scoring.

The system supports two roles: the Admin / Scorer, who manages teams, players and matches and records every delivery, and Viewers, who follow the scoreboard, player details and the team leaderboard. The scorer can create teams and players, schedule a match between two teams, and score each ball by entering runs, extras (wide, no-ball, bye, leg-bye) and wickets.

The application automatically calculates total runs, wickets, overs, strike rotation and the wicket limit of a team (total players minus one). It supports two innings with target display and an End Innings control, decides the winner after the second innings, and updates matches played, won, lost and points (two points for a win) on a team leaderboard. Every delivery is stored in a Ball record, so complete match history is preserved.

The system is implemented using Django (Python) with the Model-View-Template architecture, SQLite as the database, and HTML, CSS, Bootstrap and JavaScript for a responsive interface. Django authentication, CSRF protection and password validators secure the application, and the Django Admin panel allows teams, players, matches and balls to be managed. The project was developed following the Waterfall Model and tested using functional, category-partition, state-based, unit, integration and user acceptance testing.

Keywords: Cricket Scoring, Live Score Management, Django, SQLite, Bootstrap, Ball-by-Ball Scoring, Leaderboard, Match Management

ACKNOWLEDGEMENT

I would like to express my profound gratitude and sincere appreciation to all those who have contributed to the successful completion of the project report titled **"Live Cricket Score Management System"** as part of the project course curriculum.

First and foremost, I extend my deep gratitude to my Project Guide, Asst. Prof. Saba Ansari, for continuous guidance, valuable technical insights, constructive feedback and encouragement throughout the software development lifecycle. Her mentorship was pivotal in resolving design challenges and shaping the scope of this project.

I am immensely grateful to Asst. Prof. Saba Ansari, Head of the Department of Computer Technology, for providing the necessary laboratory infrastructure, software development tools, and an environment conducive to learning and research.

My sincere thanks also go to Dr. Shraddha Bhome, Principal of J. K. College of Science and Commerce (Autonomous), for providing the institutional support and facilities required to execute this project successfully.

I would also like to acknowledge the faculty members and technical staff of the Computer Technology department for their direct and indirect support during the design, testing and documentation phases.

Special thanks to my peers and classmates for their valuable suggestions during the testing phase and for engaging in critical discussions that helped refine the system's user interface and performance.

Lastly, I owe a heartfelt debt of gratitude to my family for their unconditional love, patience and constant encouragement throughout the course of my academic journey.

Tejas Vijay Modke
Roll No.: 2026012040 (510515147)
TYCS
J. K. College of Science and Commerce (Autonomous)

DECLARATION

I hereby declare that the project entitled **"Live Cricket Score Management System"** has been developed by me as part of my academic curriculum. This project has not been duplicated or submitted to any other university or institution for the award of any degree or diploma.

To the best of my knowledge, this project is my original work and has been completed under the guidance of Asst. Prof. Saba Ansari.

The project is done in partial fulfillment of the requirements for the award of the degree of BACHELOR OF SCIENCE (COMPUTER SCIENCE) to be submitted as a fifth semester project as a part of the curriculum.

Tejas Vijay Modke
Roll No.: 2026012040 (510515147)
TYCS
J. K. College of Science and Commerce (Autonomous)

Name and Signature of the Student

TABLE OF CONTENTS

1. INTRODUCTION..... 1
1.1 Background..... 1
1.2 Objectives..... 2
1.3 Purpose, Scope, and Applicability..... 2
1.4 Achievements..... 3
2. SURVEY OF TECHNOLOGIES..... 4
2.1 HTML5 and CSS3..... 4
2.2 Bootstrap..... 4
2.3 JavaScript..... 4
2.4 Python..... 5
2.5 Django..... 5
2.6 Django Authentication and Admin Panel..... 5
2.7 SQLite and MySQL..... 5
2.8 Visual Studio Code and PyCharm..... 5
2.9 Git and GitHub..... 5
3. REQUIREMENTS AND ANALYSIS..... 6
3.1 Problem Definition..... 6
3.2 Requirement Specifications..... 6
3.3 Planning and Scheduling..... 8
3.4 Software and Hardware Requirements..... 9
3.5 Preliminary Product Description..... 9
3.6 Conceptual Models / Feasibility Study..... 10
4. SYSTEM DESIGN..... 12
4.1 Basic Modules..... 12
4.2 Data Design..... 12
4.3 Procedural Design..... 15
4.4 User Interface Design..... 22
4.5 Security Issues..... 23
4.6 Test Cases Design..... 23
5. IMPLEMENTATION AND TESTING..... 25
5.1 Test Reports..... 25
5.2 Implementation Approaches..... 25
5.3 Coding and Testing..... 25
5.4 Code Efficiency / Testing Approach..... 31
5.5 Modifications and Improvements..... 32
5.6 Test Cases..... 32
6. RESULTS..... 34
6.1 Home Page..... 34
6.2 Registration Page..... 35
6.3 Login Page..... 35
6.4 Login Successful Page..... 35
6.5 Matches Page..... 36
6.6 Team Page..... 36
6.7 Players Page..... 37

6.8 Live Scoring Panel.....37
6.9 Leaderboard Page.....38
6.10 Admin Page.....39
7. CONCLUSIONS..... 40
7.1 Conclusion.....40
REFERENCES..... 42

LIST OF TABLES

Sr. No.	Table	Table Name / Description
1	Table 2.1	Technology Stack
2	Table 3.1	Functional Requirements
3	Table 3.2	Non-Functional Requirements
4	Table 3.3	Software Requirements
5	Table 3.4	Hardware Requirements
6	Table 4.1	Basic Modules
7	Table 4.2	Main Database Tables
8	Table 4.3	Team Table
9	Table 4.4	Player Table
10	Table 4.5	Match Table
11	Table 4.6	Ball Table
12	Table 4.7	Security Issues and Controls
13	Table 4.8	Test Cases Design
14	Table 5.1	Major Source Components
15	Table 5.2	Test Cases and Results
16	Table 6.1	Functional and Integration Test Report Summary

LIST OF FIGURES

Sr. No.	Figure	Figure Name
1	Figure 3.1	Project Gantt Chart
2	Figure 3.2	Conceptual Model of the System
3	Figure 4.1	Use Case Diagram
4	Figure 4.2	Class Diagram
5	Figure 4.3	ER Diagram
6	Figure 4.4	DFD Level 0 and DFD Level 1
7	Figure 4.5	Sequence Diagram - Recording a Ball
8	Figure 4.6	Activity Diagram - Match Scoring Workflow
9	Figure 4.7	State Diagram - Match Lifecycle
10	Figure 6.1	Home Page
11	Figure 6.2	Registration Page
12	Figure 6.3	Login Page
13	Figure 6.4	Registration / Login Successful Page
14	Figure 6.5	Matches Page
15	Figure 6.6	Team Page
16	Figure 6.7	Players Page
17	Figure 6.8	Live Scoring Panel
18	Figure 6.9	Leaderboard Page
19	Figure 6.10	Django Admin Page

1. INTRODUCTION

Cricket is one of the most widely played sports in India, and matches are held at every level, from school tournaments and society leagues to academies and practice sessions. In most of these matches, scoring is still done manually on paper scorebooks, which is slow, prone to calculation mistakes, and difficult to preserve for later reference.

Live Cricket Score Management System is a web-based application developed to provide a structured and reliable platform for managing cricket teams, players, matches and ball-by-ball scoring. The application supports two kinds of users: the **Admin / Scorer**, who creates teams and players, schedules matches and records every delivery, and **Viewers**, who follow the scoreboard, player details and the team leaderboard.

The system allows a scorer to create teams, add players, schedule a match between two teams, and score every ball by entering runs, extras (wide, no-ball, bye, leg-bye) and wickets. Total runs, overs, wickets, strike rotation, innings change and the final winner are calculated automatically, which reduces human error and speeds up the scoring process.

A modular design has been followed throughout: Team and Player Management, Match Setup, Live Scoring, Innings Handling, Scoreboard and Results, Leaderboard, and Admin Data Management operate as interconnected modules. The application is implemented using Django (Python), SQLite, HTML, CSS, Bootstrap and JavaScript.

1.1 Background

Traditionally, cricket scoring was performed using paper scorebooks or whiteboards. Scorers wrote runs, wickets, overs and extras by hand and calculated totals, run rate and strike rate manually. This process is time-consuming, error-prone, and makes long-term record keeping difficult, because past scorecards are scattered across notebooks and are rarely analysed.

Popular platforms such as CricHeroes, CricClubs and ESPNcricinfo have shown the value of digital scoring: instant updates, ball-by-ball records, player statistics and match summaries. However, many such platforms are commercial, feature-heavy or not easily customised for small tournaments, college matches and local clubs.

Live Cricket Score Management System was therefore designed as a simplified, customisable scoring application suitable for educational use, academies and local clubs. Users create teams and players, schedule a match, perform live scoring and let the system update totals, strike rotation, bowler details, winner and the points table automatically.

The application is built on the Django framework, which follows the Model-View-Template (MVT) architecture, and stores teams, players, matches and every delivery in a relational SQLite database. Django authentication protects the scoring operations, while Bootstrap provides a responsive interface that works on mobiles, tablets and desktops.

1.2 Objectives

The main objective of this project is to develop a simple, accurate and user-friendly web application for managing cricket teams, matches and live scores.

The specific objectives are:

1. To provide secure registration, login and logout for users of the system.
2. To allow the scorer to create, edit and delete teams, and to add players with name, jersey number and role.
3. To allow the scorer to schedule a match by selecting two teams.
4. To provide ball-by-ball scoring for runs, extras (wide, no-ball, bye, leg-bye) and wickets.
5. To update total runs, wickets, overs, strike rotation and bowler details automatically.
6. To support two innings, display the target for the second innings, and allow the innings to be ended.
7. To decide the winner automatically and update matches played, won, lost and points of each team.
8. To maintain a team leaderboard (2 points for a win) and a list of all players with individual summaries.
9. To store all teams, players, matches and ball records in a database for future reference.
10. To provide a responsive, clean interface and a Django Admin panel for data management.

1.3 Purpose, Scope, and Applicability

1.3.1 Purpose

The purpose of the project is to automate manual cricket scoring and replace paper scorebooks with a digital solution. The application keeps teams, players, live scores, match results and the leaderboard in a single system, so that scorers can enter ball-by-ball data and immediately see the updated score, match progress and standings.

Another purpose is transparency and record keeping: completed matches and their results remain available in the system, so that organisers and players can review them at any time.

Live Cricket Score Management System is a scoring and match-management tool. It does not replace the umpires, the official laws of cricket, or professional broadcasting scoring systems.

1.3.2 Scope

The current scope of the system includes:

1. Registration, login, logout and authentication-based access to the application.
2. Team management: create, edit and delete teams with total players and overs.
3. Player management: add, edit and delete players under a team with number and role (Batsman, Bowler, All-rounder).

4. Match scheduling by selecting Team 1 and Team 2, and deletion of matches.
5. Live ball-by-ball scoring with runs, extras and wickets, striker / non-striker / bowler selection and strike rotation.
6. Innings handling for innings 1 and 2, with target display and End Innings control.
7. Automatic winner decision and leaderboard (played, won, lost, points).
8. Player summary page and match summary on the Matches page.
9. Django Admin panel for managing users, teams, players, matches and balls.

The system focuses on cricket scoring and does not include live video streaming, tournament fixtures or knockout brackets, advanced analytics, or online sharing in its present version.

1.3.3 Applicability

The system can be used by:

- School and college cricket tournaments.
- Local clubs, societies and cricket academies.
- Scorers and tournament organisers who need quick and error-free score recording.
- Players, coaches and viewers who want to follow the scoreboard and standings.
- Educational environments for demonstrating Django, database design and software engineering concepts.

With cloud database integration, multi-user roles and live sharing, the system can be extended for larger tournaments.

1.4 Achievements

The major achievements of the project are:

1. Developed a Django and Bootstrap web application with SQLite database for cricket scoring.
2. Implemented authentication with registration, login and logout, and a Django Admin panel.
3. Implemented team and player management with edit and delete operations.
4. Implemented match scheduling and a live scoring panel with runs 0-6, Wide, No Ball, Bye, Leg Bye and Wicket controls.
5. Implemented automatic over progression, strike rotation, wicket limit (total players - 1) and bowler / batsman selection.
6. Implemented two-innings handling with target calculation and the End Innings control.
7. Implemented automatic winner decision and a points-based team leaderboard.
8. Implemented ball-level storage (Ball model) so that complete match history is preserved.
9. Designed a responsive green-themed interface using Bootstrap 5 and tested the system using functional, category-partition, state-based, unit, integration and user acceptance testing.

2. SURVEY OF TECHNOLOGIES

The selection of technologies for the Live Cricket Score Management System was based on the requirements of web application development, fast and accurate score updates, reliable data storage, security, maintainability and ease of development. Several alternatives such as PHP, Flask, Node.js, MySQL and cloud databases were considered. The following technologies and tools were selected because they suit the scope and architecture of the proposed system.

Table 2.1: Technology Stack

Sr. No.	Technology / Tool	Category
1	HTML5	Page Structure
2	CSS3	Styling
3	Bootstrap 5.3.3	Responsive UI Framework
4	JavaScript	Client-side Interactivity
5	Python 3.x	Programming Language
6	Django 5.2.6	Web Framework (MVT)
7	Django Authentication	Login, Registration, Sessions
8	Django Admin	Data Management Panel
9	SQLite	Relational Database
10	MySQL (optional)	Database for Larger Deployments
11	Visual Studio Code / PyCharm	Development Environment
12	Git and GitHub	Version Control

2.1 HTML5 and CSS3

HTML5 was selected to create the structure of all pages such as the home page, teams, players, matches, scoring panel and leaderboard. CSS3 is used for layout, colours and visual styling. Together they are lightweight, supported by all browsers and require no additional software on the user side.

2.2 Bootstrap

Bootstrap 5.3.3 was selected for building a responsive and professional interface. Its grid system, navbar, cards, buttons, alerts and badges make the application usable on mobiles, tablets and desktops, which is important because scorers often use a phone at the ground. It is loaded from a CDN in the base template.

2.3 JavaScript

JavaScript is used for interactive behaviour such as collapsible navigation, dismissible alert messages, pop-ups for selecting batsmen and bowlers, and form validation. Bootstrap's JavaScript bundle provides the navbar toggle and alert-dismiss behaviour.

2.4 Python

Python was selected because of its simple and readable syntax. The scoring rules, winner decision and leaderboard logic are written in clear Python methods in the models, which makes the project easy to understand, test and maintain.

2.5 Django

Django 5.2.6 was selected as the back-end framework. It follows the Model-View-Template (MVT) architecture, which separates data, business logic and presentation. Django provides built-in URL routing, form handling, template engine, session management, CSRF protection and an ORM, which reduces manual coding and improves reliability.

2.6 Django Authentication and Admin Panel

Django's authentication system provides secure user registration, login, logout, password hashing and password validators. The built-in Admin panel allows teams, players, matches and balls to be viewed and corrected without writing extra code, which is useful for organisers.

2.7 SQLite and MySQL

SQLite is the default Django database. It is file-based, requires no server setup and is ideal for development, demonstrations and small tournaments. Because the project uses the Django ORM, the database can be switched to MySQL for larger deployments by changing only the configuration.

2.8 Visual Studio Code and PyCharm

Visual Studio Code and PyCharm provide syntax highlighting, debugging, integrated terminal and extensions for Python and Django, which makes development and testing efficient.

2.9 Git and GitHub

Git and GitHub are used for source-code version control, change tracking and backup of the project repository.

3. REQUIREMENTS AND ANALYSIS

Requirement analysis defines the problem, identifies the users and their needs, specifies functional and non-functional requirements, and evaluates whether the proposed solution is feasible.

Live Cricket Score Management System is designed as a web application for digitalising cricket scoring. It supports the **Admin / Scorer**, who manages teams, players, matches and live scoring, and **Viewers**, who only see the updated scores, players and leaderboard. Users expect a simple interface, fast performance and error-free calculations.

3.1 Problem Definition

Cricket scoring in local matches, tournaments and practice sessions still relies on manual scorebooks, paper sheets and human calculation. Scorers write runs, wickets, overs and extras by hand, which causes delays and errors. Although some digital scoring applications exist, many are too advanced, paid, or not customisable for small tournaments.

The main problems of the existing system are:

1. Manual scorekeeping errors in run counting, over calculation and wicket updates.
2. No centralised platform to manage teams, players, matches and scoring together.
3. Slow and non-interactive scoring, with no pop-ups for selecting batsmen and bowlers and no automatic calculation.
4. No real-time score display for players, viewers and organisers.
5. Time-consuming calculation of totals, overs, run rate and extras.
6. No stored match history or team standings, which makes comparison and analysis difficult.

Therefore, there is a need for a system that can:

1. Create and manage teams and players.
2. Schedule matches between two teams.
3. Record every delivery with runs, extras and wickets.
4. Automatically update totals, overs, strike rotation and innings.
5. Decide the winner and maintain a points table.
6. Store all records permanently for later viewing.

3.2 Requirement Specifications

The system requirements are classified into functional requirements and non-functional requirements.

3.2.1 Stakeholders

- **Admin / Scorer:** a logged-in user who creates teams and players, schedules matches, records live scores, ends innings and manages data.

- **Viewer:** a user who views the scoreboard, matches, players and the team leaderboard.
- **Organiser / Coach:** a person who uses the match results and leaderboard for tournament decisions.

3.2.2 Functional Requirements

Table 3.1: Functional Requirements

ID	Functional Requirement
FR-01	The system shall provide registration, login and logout for users.
FR-02	The system shall allow the Admin to create, edit and delete teams, storing team name, total players and overs.
FR-03	The system shall automatically calculate the wicket limit of a team as total players minus one.
FR-04	The system shall allow the Admin to add, edit and delete players under a team with number, name and role.
FR-05	The system shall allow the Admin to schedule a match by selecting Team 1 and Team 2.
FR-06	The system shall allow the Admin to select the striker, non-striker and bowler for live scoring.
FR-07	The system shall allow ball-by-ball entry of runs (0, 1, 2, 3, 4, 6), extras (Wide, No Ball, Bye, Leg Bye) and wickets.
FR-08	The system shall treat wides and no-balls as illegal deliveries and update overs only for legal deliveries.
FR-09	The system shall rotate the strike on odd runs and at the end of an over.
FR-10	The system shall display total runs, wickets, overs, striker, non-striker and bowler on the scoring panel.
FR-11	The system shall support two innings and display the target during the second innings.
FR-12	The system shall provide an End Innings control.
FR-13	The system shall decide the winner automatically and update matches played, won, lost and points.
FR-14	The system shall display a team leaderboard ranked by points.
FR-15	The system shall display all players and a player summary page.
FR-16	The system shall store all teams, players, matches and ball records in the database.
FR-17	The system shall allow matches to be deleted by the Admin.
FR-18	The system shall provide a Django Admin panel for managing stored data.

3.2.3 Non-Functional Requirements

Table 3.2: Non-Functional Requirements

Requirement	Specification
Usability	The interface shall be clean and simple so that scoring can be done quickly without confusion.

Requirement	Specification
Performance	Pages shall load quickly and every ball update shall be reflected without noticeable delay.
Security	Scoring and data-management operations shall be accessible only to authenticated users; passwords shall be stored in hashed form and forms shall be CSRF-protected.
Reliability	Scoring calculations shall be accurate and consistent, and the data shall persist in the database.
Portability	The application shall work in Chrome and Firefox on mobile, tablet and desktop screens.
Maintainability	The code shall be modular (models, views, templates) and easy to extend.
Scalability	The database design shall support additional teams, matches and features, and a move to MySQL if required.
Availability	The application depends on the Django server and database being available; internet is needed only for CDN-hosted Bootstrap files.

3.3 Planning and Scheduling

The project was planned in phases: requirement analysis, system design, implementation, testing, deployment and maintenance, following the Waterfall Model. The Gantt chart shows both the planned and the actual schedule of these phases over four months (July to October), measured in weeks.

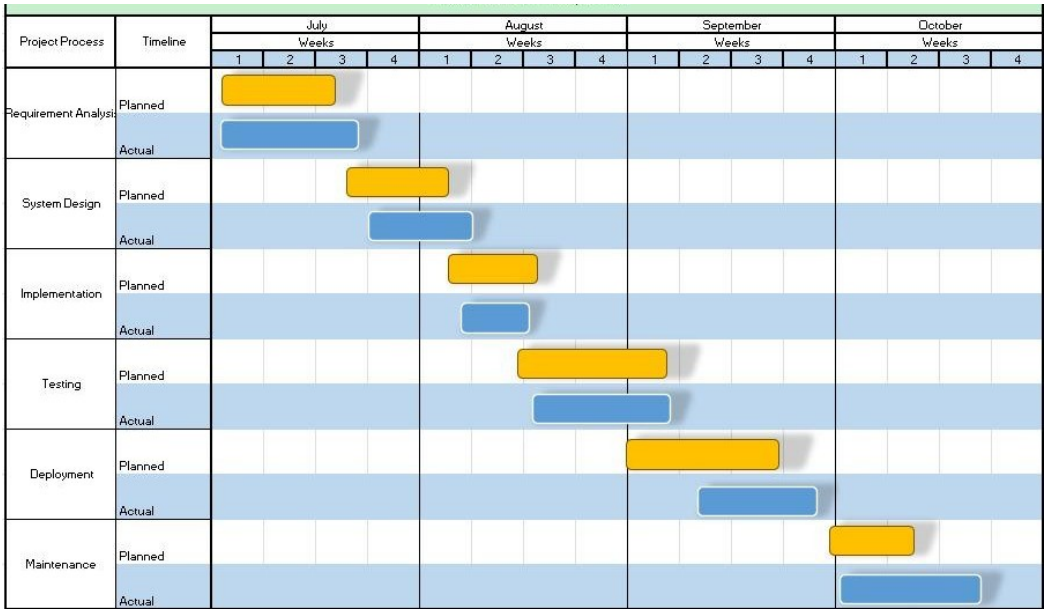


Figure 3.1: Project Gantt Chart

3.4 Software and Hardware Requirements

3.4.1 Software Requirements

Table 3.3: Software Requirements

Software / Technology	Requirement
Operating System	Windows 10 / 11 or Linux
Front-End	HTML5, CSS3, Bootstrap 5, JavaScript
Back-End	Python 3.x with Django 5.2.6
Database	SQLite (default); MySQL optional for larger data
Development Environment	Visual Studio Code / PyCharm
Web Browser	Google Chrome / Mozilla Firefox / Microsoft Edge
Version Control	Git and GitHub / GitLab (optional)
Internet Connectivity	Required to load CDN-hosted Bootstrap files, and for deployment and updates

3.4.2 Hardware Requirements

Table 3.4: Hardware Requirements

Component	Requirement
Processor	Intel i3 or above
RAM	Minimum 4 GB
Hard Disk	At least 10 GB free space
Monitor	15" or above
Input Devices	Keyboard and Mouse
User Device	Mobile, tablet or computer with a modern web browser

3.5 Preliminary Product Description

Live Cricket Score Management System is a web application that provides structured management of cricket teams, players, matches and live scoring. The application has an Admin / Scorer role that manages and scores matches, and Viewers who follow the results. It follows the Django MVT architecture, with SQLite as the database and Bootstrap for the user interface.

The major product capabilities include:

1. User registration, login and logout.
2. Team creation with total players and overs; automatic wicket limit.
3. Player management with number, name and role (Batsman / Bowler / All-rounder).
4. Match scheduling between any two teams.
5. Live scoring panel with runs, extras, wicket and End Innings controls.

6. Automatic updating of total score, overs, strike rotation and target.
7. Automatic winner decision and leaderboard update.
8. Player summary and match history display.
9. Django Admin panel for data management.

3.6 Conceptual Models / Feasibility Study

3.6.1 Conceptual Model

The conceptual model shows the major flow of the system. The Admin / Scorer logs in, creates teams and players, and schedules a match. During the match the scorer opens the Live Score Control and records each ball. The system updates the score, overs, strike and innings automatically and stores every delivery in the database. At the end of the match the winner is decided and the leaderboard is updated, and Viewers can see the results.

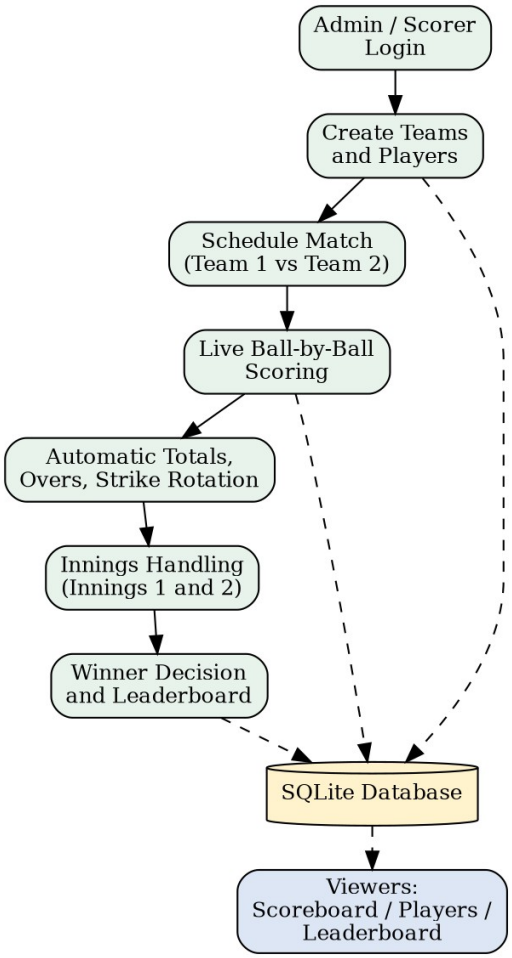


Figure 3.2: Conceptual Model of the System

3.6.2 Feasibility Study

A. Technical Feasibility

The system is technically feasible because Python, Django, SQLite, Bootstrap and JavaScript provide all the capabilities required for authentication, data storage, scoring logic and a responsive interface. They are mature, well documented and run on ordinary computers.

B. Economic Feasibility

The project is economically feasible because all tools used (Python, Django, SQLite, Bootstrap, Visual Studio Code, Git) are free and open source. No paid server, licence or specialised hardware is required for development or demonstration.

C. Operational Feasibility

The system is operationally feasible because its interface uses familiar web interactions such as forms, buttons, cards and tables. Scorers need only basic knowledge of cricket scoring. The separation of Admin and Viewer responsibilities keeps the workflow simple, and the automatic calculations reduce the effort and errors of manual scoring.

4. SYSTEM DESIGN

System design converts the identified requirements into an organised technical solution. The Live Cricket Score Management System follows a modular web architecture based on the Django Model-View-Template (MVT) pattern. Team management, player management, match setup, live scoring, innings handling, results and leaderboard operate as interconnected modules that share a single relational database.

The system supports an Admin / Scorer, who must be authenticated to change data, and Viewers, who read the scoreboard, players and leaderboard. Django authentication handles identity and sessions, the Django ORM stores application data in SQLite, and Bootstrap templates render the interface.

4.1 Basic Modules

Table 4.1: Basic Modules

Sr. No.	Module	Purpose	Major Functions
1	Authentication and Access	Provides secure access to the application.	Register, login, logout, Django Admin login
2	Team Management	Creates and maintains teams.	Add, edit, delete team; total players, overs, wicket limit
3	Player Management	Maintains team rosters.	Add, edit, delete player; number, name, role; player summary
4	Match Setup	Configures a match before scoring.	Select Team 1 vs Team 2, schedule and delete match
5	Live Scoring	Records every delivery in real time.	Runs, extras, wicket, striker / non-striker / bowler selection, strike rotation, over progression
6	Innings Handling	Controls the two innings of a match.	Target display, End Innings, switch batting team
7	Scoreboard and Results	Shows progress and outcome.	Totals, overs, wickets, match summary, winner decision
8	Leaderboard	Maintains team standings.	Matches played / won / lost, points (2 per win), ranking
9	Admin Data Management	Maintains all stored data.	Django Admin for users, teams, players, matches, balls
10	User Interface	Provides a responsive interface.	Bootstrap navbar, cards, alerts, forms, mobile layout

4.2 Data Design

The application uses SQLite, a relational database, accessed through the Django ORM. Each Django model is mapped to a database table, and relationships between entities are

implemented using foreign keys. The authentication tables (user, group, session) are provided by Django itself.

Table 4.2: Main Database Tables

Table (Model)	Purpose
home_team (Team)	Stores teams, wicket limit and leaderboard statistics.
home_player (Player)	Stores players belonging to a team.
home_match (Match)	Stores matches, innings scores, winner and the live scoring state.
home_ball (Ball)	Stores every delivery of a match (ball-by-ball record).
auth_user (User)	Stores registered users with hashed passwords (Django authentication).

4.2.1 Data Dictionary

Every model also has an automatically generated primary key field **id** (BigAutoField). The tables below describe the fields defined in the project.

Table 4.3: Team Table

Field Name	Data Type	Constraint	Description
id	BigAutoField	Primary key	Unique team identifier
name	CharField(100)	Unique	Team name (e.g., MI, CSK)
total_players	IntegerField	Required	Number of players in the team
overs	IntegerField	Required	Overs per innings
wickets	IntegerField	editable = False	Wicket limit, calculated as total_players - 1
disqualified	BooleanField	Default False	Whether the team is disqualified
matches_played	PositiveIntegerField	Default 0	Matches played
matches_won	PositiveIntegerField	Default 0	Matches won
matches_lost	PositiveIntegerField	Default 0	Matches lost
points	PositiveIntegerField	Default 0	Leaderboard points (2 per win)

Table 4.4: Player Table

Field Name	Data Type	Constraint	Description
id	BigAutoField	Primary key	Unique player identifier
team	ForeignKey(Team)	CASCADE	Team of the player (related name: players)
number	IntegerField	Required	Jersey number
name	CharField(100)	Required	Player name
player_type	CharField(20)	Choices	batsman / bowler / allrounder

Table 4.5: Match Table

Field Name	Data Type	Constraint	Description
id	BigAutoField	Primary key	Unique match identifier
team1, team2	ForeignKey(Team)	CASCADE	The two teams of the match
date	DateTimeField	auto_now_add	Date and time of match creation
team1_runs, team2_runs	PositiveIntegerField	Default 0	Runs scored by each team
team1_overs, team2_overs	DecimalField(4,1)	Default 0.0	Overs played (e.g., 9.3)
team1_wickets, team2_wickets	PositiveIntegerField	Default 0	Wickets lost by each team
winner	ForeignKey(Team)	SET_NULL, null	Winning team (empty until decided)
current_striker, current_non_striker, current_bowler	ForeignKey(Player)	SET_NULL, null	Live scoring state of the players
current_batting_team	ForeignKey(Team)	SET_NULL, null	Team currently batting
innings	IntegerField	Default 1	Current innings (1 or 2)

Table 4.6: Ball Table

Field Name	Data Type	Constraint	Description
id	BigAutoField	Primary key	Unique delivery identifier
match	ForeignKey(Match)	CASCADE	Match to which the ball belongs (related name: balls)
innings	PositiveSmallIntegerFiel d	Default 1	Innings number
bowler, striker, non_striker	ForeignKey(Player)	SET_NULL, null	Players involved in the delivery
over_number, ball_in_over	IntegerField	Required	Position of the ball in the innings
runs	IntegerField	Default 0	Runs on this delivery
is_legal_delivery	BooleanField	Default True	False for wide and no-ball
is_wicket	BooleanField	Default False	True when a wicket falls
extra_type	CharField(10)	Choices, null	wide / noball / bye / legbye
player_out	ForeignKey(Player)	SET_NULL, null	Batsman dismissed (only if is_wicket)

4.2.2 Data Integrity and Constraints

- **Unique team name:** the name field of Team is unique, so duplicate teams cannot be created.
- **Derived wicket limit:** wickets is not editable and is always calculated as total_players - 1 when a team is saved.

- **Referential integrity:** Player and Ball depend on Team and Match through foreign keys; deleting a team or match cascades to its players or balls.
- **Safe deletion of references:** live-state fields, winner and ball participants use SET_NULL so that removing a player does not break stored matches.
- **Non-negative counters:** runs, wickets and statistics use PositiveIntegerField.
- **Controlled choices:** player_type and extra_type accept only predefined values.
- **Single decision of winner:** decide_winner() returns immediately if a winner already exists, so leaderboard statistics are not counted twice.

4.3 Procedural Design

4.3.1 Logic Diagrams

4.3.1.1 Use Case Diagram

The use case diagram shows the interaction of the Admin / Scorer and the Viewer with the system.

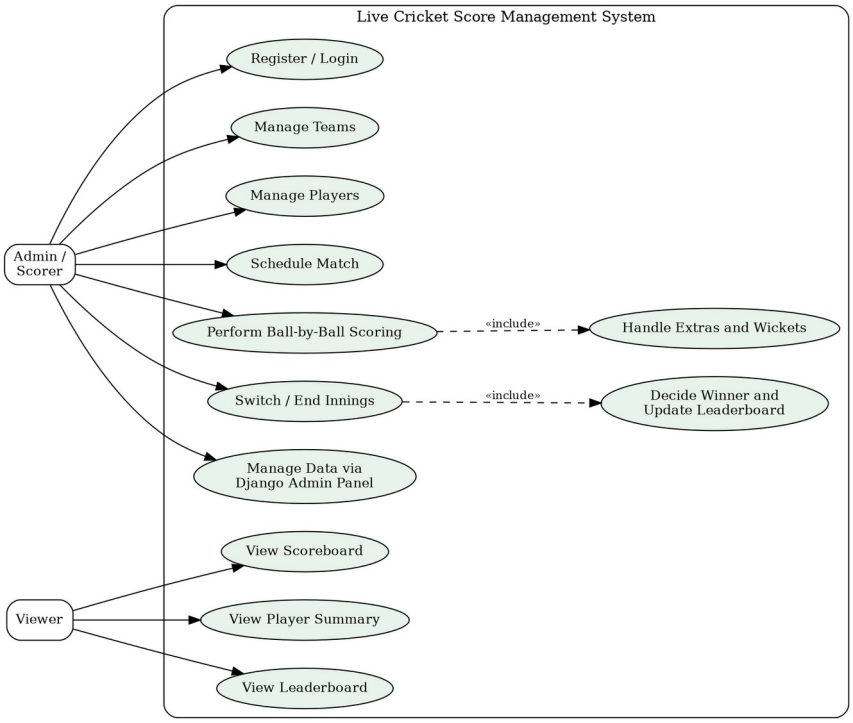


Figure 4.1: Use Case Diagram

4.3.1.2 Class Diagram

The class diagram shows the four main model classes, their attributes, methods and relationships.

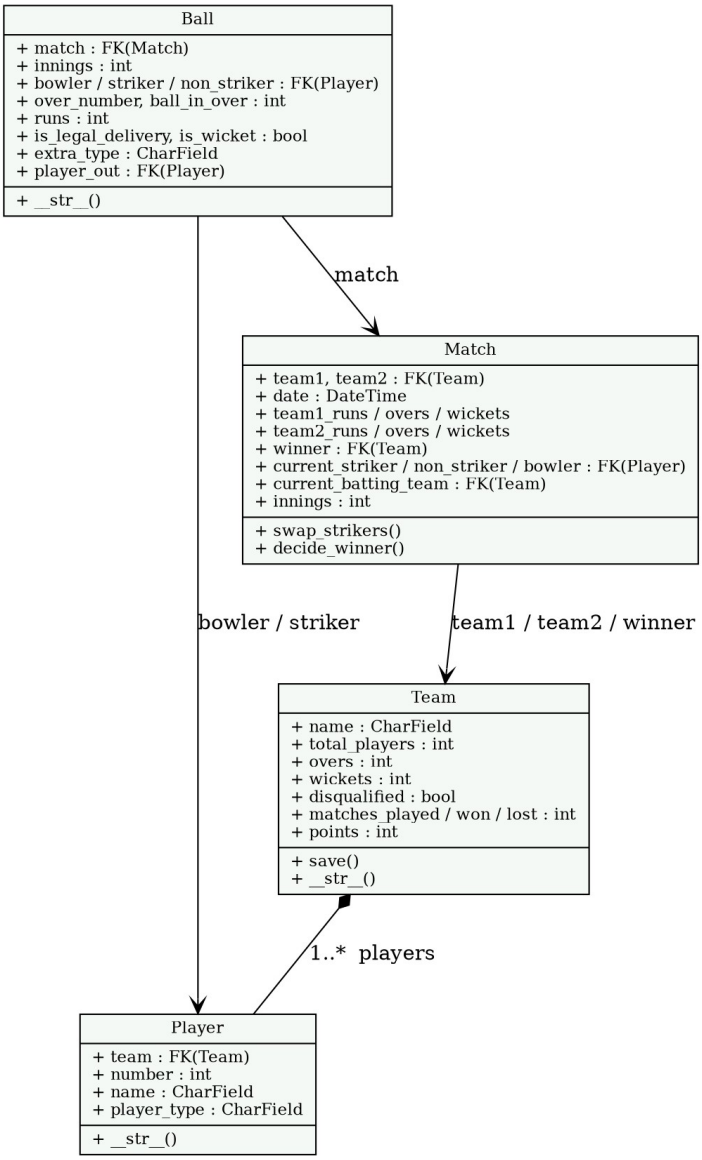


Figure 4.2: Class Diagram

4.3.1.3 ER Diagram

The Entity-Relationship diagram shows that one Team has many Players, a Team takes part in many Matches, a Match has many Ball records, and Players are linked to Ball records as bowler, striker, non-striker or dismissed player.

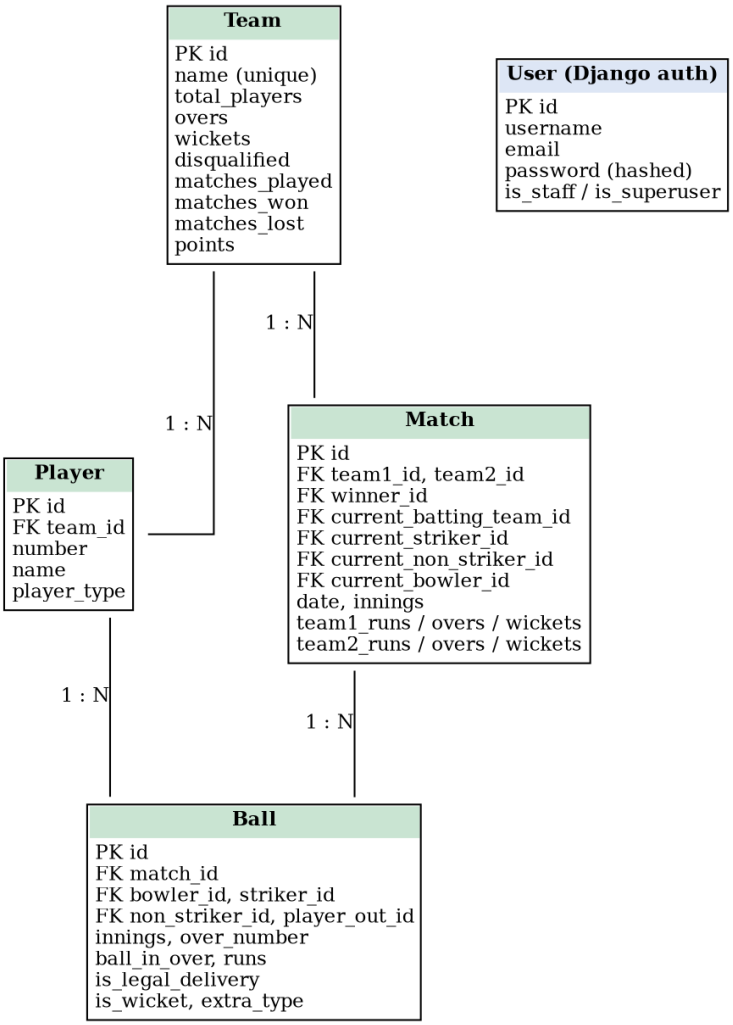


Figure 4.3: ER Diagram

4.3.1.4 DFD Level 0 and Level 1

At Level 0 the Admin exchanges login information, team details, match configuration and ball data with the system and receives scoreboard display, error messages and winner status. At Level 1 the system is divided into the processes Authenticate User, Manage Teams, Match Setup and Scoring Engine, which use the User, Player, Team, Match, Ball-by-Ball and Player Status data stores.

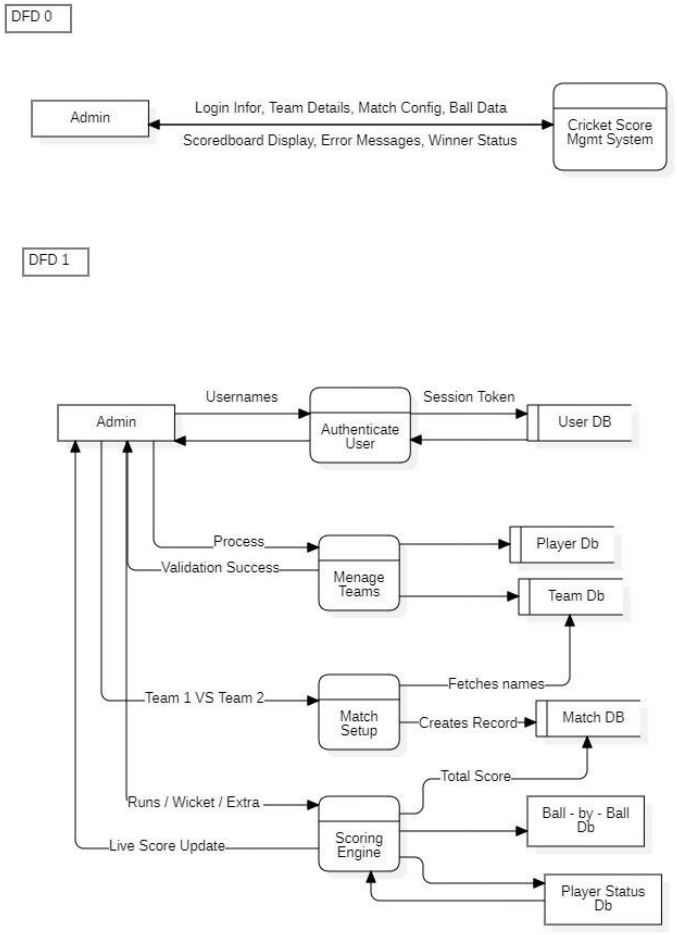


Figure 4.4: DFD Level 0 and DFD Level 1

4.3.1.5 Sequence Diagram

The sequence diagram shows the flow of a single ball entry from the scorer to the database and back to the scoreboard.

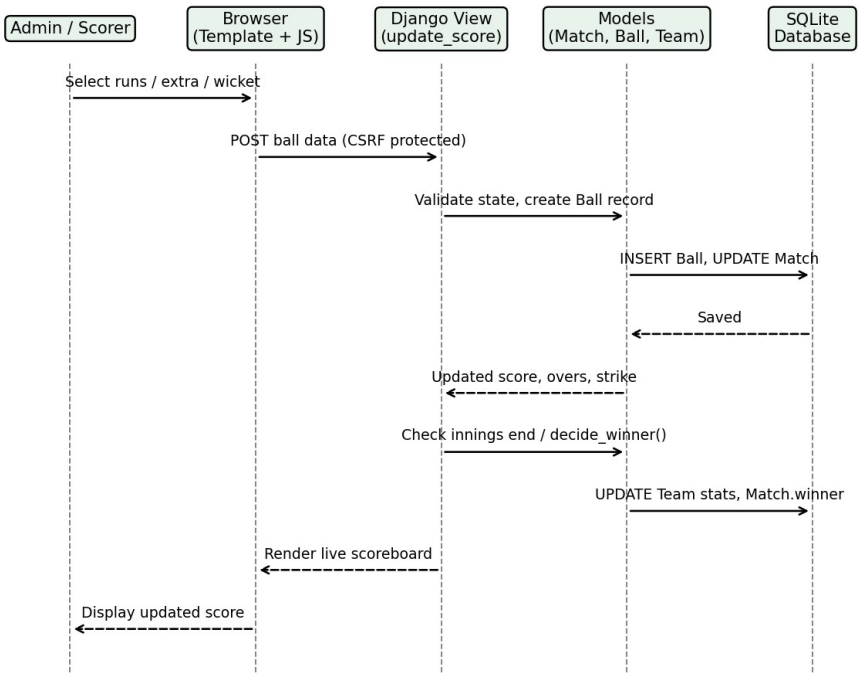


Figure 4.5: Sequence Diagram - Recording a Ball

4.3.1.6 Activity Diagram

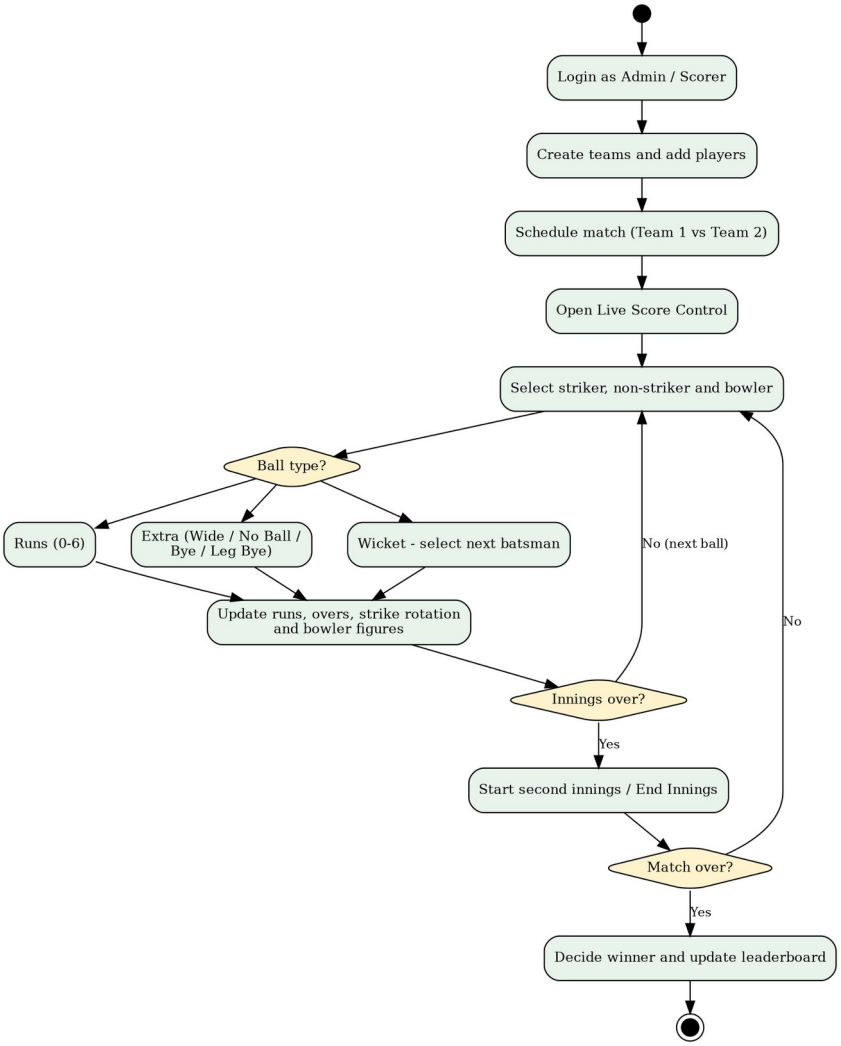


Figure 4.6: Activity Diagram - Match Scoring Workflow

4.3.1.7 State Chart Diagram

A match moves through the states Scheduled, First Innings, Innings Break, Second Innings and Completed.

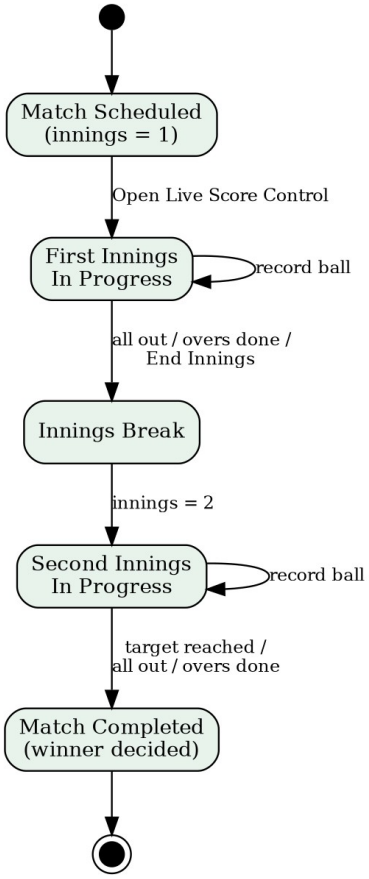


Figure 4.7: State Diagram - Match Lifecycle

4.3.2 Data Structures

The following main data structures are used by the application:

- **Model instances (Team, Player, Match, Ball):** Django ORM objects mapped to database tables.
- **QuerySets:** used to fetch teams, players, matches and ordered leaderboard data.
- **Choice tuples:** PLAYER_TYPES and extra_type choices restrict input to valid values.
- **Session and message framework:** stores the logged-in user and flash messages such as "Welcome back".
- **Live state fields in Match:** current_striker, current_non_striker, current_bowler, current_batting_team and innings hold the state of the match being scored.

4.3.3 Algorithm Design

A. Wicket Limit Calculation

1. Read total_players of the team.
2. If total_players >= 2, set wickets = total_players - 1.
3. Save the team.

B. Ball-by-Ball Scoring

1. Scorer selects runs, an extra type or a wicket for the current delivery.

2. Create a Ball record with match, innings, bowler, striker, non-striker, over number and ball number.
3. If the extra is a wide or no-ball, mark the delivery as not legal; otherwise it is a legal delivery.
4. Add runs to the batting team total; for a wicket increase the wickets and record player_out.
5. For a legal delivery increase the ball count; after six legal balls complete the over and change the bowler.
6. If the run total is odd, or the over is completed, swap the striker and non-striker.
7. If wickets reach the team wicket limit or overs are completed, the innings ends.

C. Strike Rotation

1. Read current_striker and current_non_striker of the match.
2. Exchange the two values (swap_strikers()).
3. Save the match.

D. Winner Decision and Leaderboard Update

1. If the match already has a winner, stop.
2. Compare team1_runs and team2_runs after the second innings.
3. The team with the higher runs is the winner; otherwise there is no winner (tie).
4. Increase matches_played of both teams.
5. For the winner increase matches_won and add 2 points; for the other team increase matches_lost.
6. Save both teams and the match.

4.4 User Interface Design

The interface follows a consistent green cricket theme built with Bootstrap 5. A fixed navigation bar provides links to Home, Matches, Teams, Players and Leaderboard, together with Login / Register or the logged-in user name and Logout. A footer and dismissible alert messages are common to all pages.

- **Home page:** form to create a team (name, total players, overs) and list of created teams.
- **Register and Login pages:** simple forms for account creation and sign-in.
- **Matches page:** form to schedule a match, and match cards showing scores, winner and a Live Score Control button.
- **Teams page:** team cards with players, statistics and Edit / Delete / Add Player buttons.
- **Players page:** player cards with Edit, Summary and Delete buttons.
- **Live scoring panel:** score card with striker, non-striker, bowler and target, and buttons for runs, extras, wicket and End Innings.
- **Leaderboard page:** ranked table of teams with played, won, lost and points.
- **Admin page:** Django administration site for data management.

Screen layouts of these pages are presented in Chapter 6 (Results).

4.5 Security Issues

Table 4.7: Security Issues and Controls

Issue	Risk	Control
Unauthorised access	A user changes teams or scores without login	Django authentication and login-based access to scoring pages
Weak passwords	Easily guessed passwords	Django password validators (similarity, length, common, numeric)
Password disclosure	Passwords visible in database	Passwords stored as hashes by Django
Cross-site request forgery	Forged form submissions	CsrfViewMiddleware and CSRF tokens in forms
Clickjacking	Page embedded in a malicious frame	XFrameOptionsMiddleware
Duplicate data	Two teams with the same name	Unique constraint on Team.name
Invalid input	Letters or negative values as runs	Server-side validation and restricted run buttons (0-6)
Double counting of results	Leaderboard updated twice for one match	decide_winner() checks whether the winner is already set
Development settings	DEBUG = True and a development SECRET_KEY are unsuitable for production	Must be changed, with ALLOWED_HOSTS configured, before deployment

4.6 Test Cases Design

Table 4.8: Test Cases Design

ID	Test Scenario	Expected Result
T01	Register with valid details	Account created and user logged in
T02	Register with mismatching passwords	Registration rejected with error message
T03	Login with valid and invalid credentials	Valid: Welcome message; invalid: error shown
T04	Create team with valid data	Team added successfully
T05	Create team with blank or duplicate name	Validation error shown
T06	Team with 11 players	Wicket limit set to 10
T07	Add, edit and delete a player	Roster updated correctly
T08	Schedule match with two teams	Match appears on Matches page
T09	Schedule match with the same team twice	Rejected or flagged
T10	Enter runs 0 to 6	Score increases by the entered runs
T11	Enter invalid run (letter, negative number)	Input rejected
T12	Wide / No Ball	Run added, over count does not increase
T13	Bye / Leg Bye	Runs added as extras, ball counted

ID	Test Scenario	Expected Result
T14	Wicket entry	Wicket count increases; new batsman selected
T15	Six legal balls	Over completes, strike changes, next over starts
T16	Odd runs	Striker and non-striker swap
T17	All out / overs completed	Innings ends
T18	End Innings button	Second innings starts with correct target
T19	Second innings run chase completed	Winner decided and shown
T20	Winner decided	Played, won, lost and points updated once
T21	Open Leaderboard	Teams ranked by points
T22	Delete match	Match removed from the list
T23	Access scoring without login	User redirected or denied

5. IMPLEMENTATION AND TESTING

This chapter describes how the designed system was implemented and how it was tested. The implementation follows the Django MVT structure, and each module was developed and tested separately before integration.

5.1 Test Reports

Testing was carried out using functional testing, the category partition method, state-based testing, unit testing, integration testing and user acceptance testing. Each feature was tested with valid, invalid and boundary inputs. The detailed results are summarised in Chapter 6 (Table 6.1) and the test cases are listed in Section 5.6.

5.2 Implementation Approaches

The system was developed using a modular approach. Models were designed first, followed by URL routing, views, templates and finally integration of scoring, innings and leaderboard logic. Business rules such as the wicket limit, strike swap and winner decision are implemented as methods of the model classes so that they are reusable and easy to test.

Table 5.1: Major Source Components

Implementation Area	Source File / Path
Project settings	cricket/settings.py
Project URL configuration	cricket/urls.py
Application URL routes	home/urls.py
Data models and scoring rules	home/models.py
Views (login, teams, players, matches, scoring, leaderboard)	home/views.py
Common layout (navbar, alerts, footer)	templates/base.html
Page templates	templates/ (home, matches, teams, players, scoring, leaderboard)
Admin registration	home/admin.py

5.3 Coding and Testing

5.3.1 Project Settings

The settings file registers the home application, the security middleware (including CSRF protection), the templates and static folders, and the SQLite database.

```
# cricket/settings.py  (key configuration)
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

DEBUG = True
ALLOWED_HOSTS = []
```

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'home.apps.HomeConfig',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'cricket.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / "templates"],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                "django.template.context_processors.debug",
                "django.template.context_processors.request",
                "django.contrib.auth.context_processors.auth",
                "django.contrib.messages.context_processors.messages",
            ],
        },
    },
]

STATIC_URL = 'static/'
STATICFILES_DIRS = [BASE_DIR / "static"]
WSGI_APPLICATION = 'cricket.wsgi.application'

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

AUTH_PASSWORD_VALIDATORS = [
    {'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator'},
    {'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator'},
    {'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator'},
    {'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator'},
]

LANGUAGE_CODE = 'en-us'
TIME_ZONE = 'UTC'
USE_I18N = True
USE_TZ = True
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

5.3.2 Data Models and Scoring Rules

The Team model calculates the wicket limit automatically in its save() method. The Match model stores live scoring state and provides swap_strikers() and decide_winner(). The Ball model stores every delivery.

```
# home/models.py
from django.db import models


class Team(models.Model):
    name = models.CharField(max_length=100, unique=True)
    total_players = models.IntegerField()
    overs = models.IntegerField()
    wickets = models.IntegerField(editable=False)
    disqualified = models.BooleanField(default=False)

    # Leaderboard Statistics Fields
    matches_played = models.PositiveIntegerField(default=0)
    matches_won = models.PositiveIntegerField(default=0)
    matches_lost = models.PositiveIntegerField(default=0)
    points = models.PositiveIntegerField(default=0)

    def save(self, *args, **kwargs):
        # Auto-calculate wickets limit (total players - 1)
        if self.total_players >= 2:
            self.wickets = self.total_players - 1
        super().save(*args, **kwargs)

    def __str__(self):
        return self.name


class Player(models.Model):
    PLAYER_TYPES = (
        ('batsman', 'Batsman'),
        ('bowler', 'Bowler'),
        ('allrounder', 'All Rounder'),
    )
    team = models.ForeignKey(Team, on_delete=models.CASCADE,
                             related_name="players")
    number = models.IntegerField()
    name = models.CharField(max_length=100)
    player_type = models.CharField(max_length=20, choices=PLAYER_TYPES)

    def __str__(self):
        return f"{self.number} - {self.name} ({self.player_type})"


class Match(models.Model):
    team1 = models.ForeignKey("Team", related_name="team1_matches",
                              on_delete=models.CASCADE)
    team2 = models.ForeignKey("Team", related_name="team2_matches",
                              on_delete=models.CASCADE)
    date = models.DateTimeField(auto_now_add=True)

    # Score fields (Final/Total Score)
    team1_runs = models.PositiveIntegerField(default=0)
    team1_overs = models.DecimalField(max_digits=4, decimal_places=1, default=0.0)
    team1_wickets = models.PositiveIntegerField(default=0)
    team2_runs = models.PositiveIntegerField(default=0)
    team2_overs = models.DecimalField(max_digits=4, decimal_places=1, default=0.0)
    team2_wickets = models.PositiveIntegerField(default=0)
```

```
    winner = models.ForeignKey("Team", related_name="won_matches",
                               on_delete=models.SET_NULL, null=True, blank=True)

    # Live Scoring State Fields
    current_striker = models.ForeignKey(Player, on_delete=models.SET_NULL,
                          null=True, blank=True, related_name='striker_in')
    current_non_striker = models.ForeignKey(Player, on_delete=models.SET_NULL,
                          null=True, blank=True, related_name='non_striker_in')
    current_bowler = models.ForeignKey(Player, on_delete=models.SET_NULL,
                          null=True, blank=True, related_name='bowler_in')
    current_batting_team = models.ForeignKey(Team, on_delete=models.SET_NULL,
                          null=True, blank=True, related_name='currently_batting')

    # Status field (for tracking innings)
    innings = models.IntegerField(default=1)  # Tracks innings 1 or 2

    def __str__(self):
        return f"{self.team1.name} vs {self.team2.name}"

    def swap_strikers(self):
        """Swaps the batsmen on strike."""
        self.current_striker, self.current_non_striker = \
            self.current_non_striker, self.current_striker
        self.save()

    def decide_winner(self):
        """
        Automatically decide winner based on runs and update team
        statistics (matches played, won, lost and points).
        Runs once at the end of the match.
        """
        if self.winner:  # Only calculate if not already set
            return

        team1 = self.team1
        team2 = self.team2

        if self.team1_runs > self.team2_runs and self.innings >= 2:
            self.winner = team1
        elif self.team2_runs > self.team1_runs and self.innings >= 2:
            self.winner = team2
        else:
            self.winner = None

        if self.winner or self.innings >= 2:
            team1.matches_played += 1
            team2.matches_played += 1

            if self.winner == team1:
                team1.matches_won += 1
                team1.points += 2
                team2.matches_lost += 1
            elif self.winner == team2:
                team2.matches_won += 1
                team2.points += 2
                team1.matches_lost += 1

            team1.save()
            team2.save()
            self.save()


class Ball(models.Model):
```

```
    match = models.ForeignKey(Match, on_delete=models.CASCADE,
                              related_name="balls")
    innings = models.PositiveSmallIntegerField(default=1)

    # Who was involved
    bowler = models.ForeignKey(Player, on_delete=models.SET_NULL,
                               null=True, related_name="bowled_balls")
    striker = models.ForeignKey(Player, on_delete=models.SET_NULL,
                                null=True, related_name="faced_balls")
    non_striker = models.ForeignKey(Player, on_delete=models.SET_NULL,
                                null=True, related_name="non_faced_balls")

    # Event data
    over_number = models.IntegerField()
    ball_in_over = models.IntegerField()
    runs = models.IntegerField(default=0)

    # Event types (Wides, No-balls, Wicket, etc.)
    is_legal_delivery = models.BooleanField(default=True)  # False: wide/no-ball
    is_wicket = models.BooleanField(default=False)
    extra_type = models.CharField(max_length=10, blank=True, null=True,
        choices=[('wide', 'Wide'), ('noball', 'No Ball'),
                 ('bye', 'Bye'), ('legbye', 'Leg Bye')])

    # Player who got out (only if is_wicket is True)
    player_out = models.ForeignKey(Player, on_delete=models.SET_NULL,
                                   null=True, blank=True, related_name="got_out")

    def __str__(self):
        return (f"Match {self.match.id} | Over {self.over_number}."
                f"{self.ball_in_over} | {self.runs} Run(s)")
```

5.3.3 URL Routing

The project URLs include the application URLs. The application URLs map pages and actions such as login, teams, players, matches, live scoring, leaderboard and deletion to their views.

```
# home/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('login/', views.login_view, name='login'),
    path('register/', views.register_view, name='register'),
    path('logout/', views.logout_view, name='logout'),
    path('', views.home, name='home'),
    path('matches/', views.matches, name='matches'),
    path('teams/', views.teams, name='teams'),
    path('players/', views.players, name='players'),
    path('leaderboard/', views.leaderboard_view, name='leaderboard'),
    path('add-player/<int:team_id>/', views.add_player, name='add_player'),
    path('edit-team/<int:team_id>/', views.edit_team, name='edit_team'),
    path('delete-team/<int:team_id>/', views.delete_team, name='delete_team'),
    path('edit-player/<int:player_id>/', views.edit_player, name='edit_player'),
    path('delete-player/<int:player_id>/', views.delete_player, name='delete_player'),
    path('match-setup/<int:match_id>/', views.match_setup, name='match_setup'),
    path('match/<int:match_id>/live/', views.update_score, name='live_scoring'),
    path('update-score/<int:match_id>/', views.update_score, name='update_score'),
    path('player/<int:player_id>/summary/', views.player_summary,
name='player_summary'),
    path('delete-match/<int:match_id>/', views.delete_match, name='delete_match'),
]
```

```
# cricket/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('home.urls')),   # connects the home app URLs
]
```

5.3.4 Base Template

The base template defines the navigation bar, message alerts, content block and footer shared by every page.

```
<!-- templates/base.html (structure) -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}Live Cricket Score System{% endblock %}</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
        rel="stylesheet">
  <style> body { padding-top: 70px; } footer { border-top: 2px solid #28a745; } </style>
</head>
<body class="bg-light">
  <nav class="navbar navbar-expand-lg navbar-dark bg-success fixed-top shadow">
    <div class="container">
      <a class="navbar-brand fw-bold" href="/">Live Cricket</a>
      <ul class="navbar-nav me-auto">
        <li class="nav-item"><a class="nav-link" href="/">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="/matches/">Matches</a></li>
        <li class="nav-item"><a class="nav-link" href="/teams/">Teams</a></li>
        <li class="nav-item"><a class="nav-link" href="/players/">Players</a></li>
        <li class="nav-item"><a class="nav-link" href="{% url 'leaderboard'
%}">Leaderboard</a></li>
      </ul>
      <ul class="navbar-nav">
        {% if user.is_authenticated %}
          <li class="nav-item"><span class="nav-link text-white">
              Welcome, <strong>{{ user.username }}</strong></span></li>
          <li class="nav-item"><a class="nav-link" href="/logout/">Logout</a></li>
        {% else %}
          <li class="nav-item"><a class="nav-link" href="/login/">Login</a></li>
          <li class="nav-item"><a class="nav-link" href="/register/">Register</a></li>
        {% endif %}
      </ul>
    </div>
  </nav>

  <div class="container mt-4">
    {% if messages %}
      {% for message in messages %}
        <div class="alert alert-{{ message.tags }} alert-dismissible fade show">
          {{ message }}
        </div>
      {% endfor %}
    {% endif %}
    {% block body %}{% endblock %}
  </div>

  <footer class="bg-dark text-white text-center mt-5 py-5">
```

```
    <p>&copy; 2025 Live Cricket Score Management System</p>
    <p>Built with <strong>Django</strong> &amp; <strong>Bootstrap</strong></p>
  </footer>
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></
script>
</body>
</html>
```

5.4 Code Efficiency / Testing Approach

Testing is a crucial phase to ensure that all modules work correctly, scoring is accurate and the system meets the expectations of the Admin and viewers. The approaches used are described below.

Functional Testing

Functional testing verifies that each feature performs according to the requirements: team creation and player addition, match setup, live scoring (runs, wickets, extras, over updates), scoreboard updates, result generation and Admin operations. Each function was tested with valid, invalid and boundary-level inputs.

Category Partition Method

Input data is divided into partitions and test cases are created for every category. For score input the partitions are valid runs (0 to 6), invalid inputs (letters, symbols, negative numbers), extras (wide, no-ball, bye, leg-bye), wicket conditions and over-completion scenarios.

State-Based Testing

Some behaviour depends on the system state: match scheduled, first innings, innings break, second innings, match completed, user logged in and viewer only. Each state transition was tested, for example, no scoring after match completion and no access to Admin modules without login.

5.4.1 Unit Testing

Unit testing checks individual modules independently to find errors early and to validate normal and boundary cases.

- **Team and Player Management:** add, edit, delete team; add player; validate player details.
- **Match Setup:** create match, validate team selection, required inputs.
- **Live Scoring:** run entry (0-6), wicket entry and batsman / bowler update, extras (wide, no-ball, bye, leg-bye), over count and ball progression.
- **Scoreboard and Summary:** update totals, display batsman and bowler details, generate result and leaderboard.

Unit testing methods used were manual testing, print / log-based debugging and the Django test framework where applicable.

5.4.2 Integration Testing

Integration testing verifies that the modules work together as a unified system.

1. Team / Player Module to Match Setup: teams and players added in the database load correctly during match creation.
2. Match Setup to Live Scoring: team names, players and overs appear correctly in the scoring screen.
3. Live Scoring to Scoreboard: every ball (runs, wickets, extras) is reflected on the scoreboard.
4. Live Scoring to Result and Leaderboard: after the second innings the winner is decided and the points table is updated.
5. Admin Module to Database to UI: changes made through Django Admin appear in the user interface.

5.4.3 Beta Testing / User Validation

The system was tried by an Admin / scorer, tournament organisers and regular viewers. They checked ease of scoring, accuracy of ball-by-ball updates, smooth navigation between overs and innings, clear visibility of the scoreboard, correct final result and the absence of confusing or delayed buttons. Their feedback helped to refine the user interface, fix small calculation issues and improve the scoring flow.

5.5 Modifications and Improvements

The following improvements were made during development and testing:

- Wicket limit was changed from a manual entry to an automatic calculation (total players - 1).
- Live scoring state (striker, non-striker, bowler, batting team, innings) was moved into the Match model so that scoring can be resumed at any time.
- A separate Ball model was introduced to preserve complete ball-by-ball history.
- Winner decision and leaderboard update were centralised in decide_winner() with a guard against double counting.
- A team leaderboard and player summary pages were added.
- Navigation highlighting, alert messages and a responsive layout were added to improve usability.

5.6 Test Cases

Table 5.2: Test Cases and Results

ID	Module	Test Type	Input	Expected Output	Result
TC01	Team	Unit	Team name "Mumbai Kings"	Team added successfully	Pass
TC02	Team	Unit	Blank / duplicate name	Validation error	Pass

ID	Module	Test Type	Input	Expected Output	Result
TC03	Match	Unit	Team 1 and Team 2 selected	Match created	Pass
TC04	Scoring	Unit	Runs = 4	Score increases by 4	Pass
TC05	Scoring	Unit	Wicket = true	Wicket count increases; new batsman	Pass
TC06	Scoring	Unit	Wide / No Ball	Extra added; ball not counted	Pass
TC07	Scoring	Unit	6 legal balls	Over completes; next over starts	Pass
TC08	Result	Unit	Complete both innings	Winner and summary displayed	Pass
TC09	Integration	Integration	Admin creates match	Match appears in scoring interface	Pass
TC10	Integration	Integration	Scorer enters ball data	Scoreboard updates instantly	Pass
TC11	Integration	Integration	Innings completed	Result summary loads correctly	Pass
TC12	Integration	Integration	Admin edits team / player	Updated data appears in match setup	Pass

5.6.1 Test Result Summary

All the listed test cases produced the expected output during testing. Minor calculation and navigation issues found during user acceptance testing were corrected before the final version.

6. RESULTS

This chapter presents the output screens of the implemented Live Cricket Score Management System together with the test report summary.

Table 6.1: Functional and Integration Test Report Summary

Module	Test Type	Result
Authentication (Register / Login / Logout)	Functional	Pass
Team and Player Management	Functional, Unit	Pass
Match Setup	Functional, Integration	Pass
Live Scoring (runs, extras, wickets, overs, strike)	Functional, Category Partition, State-Based	Pass
Innings Handling and Target	State-Based, Integration	Pass
Winner Decision and Leaderboard	Functional, Integration	Pass
Admin Panel and Data Management	Integration	Pass
User Interface and Navigation	User Acceptance	Pass

6.1 Home Page

The home page contains the Create a Team form (name, total players, overs) and the list of teams created.

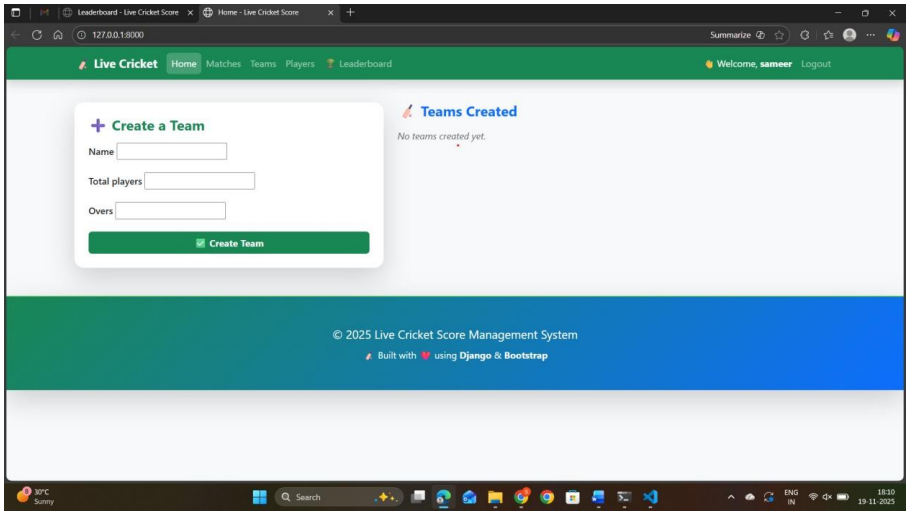


Figure 6.1: Home Page

6.2 Registration Page

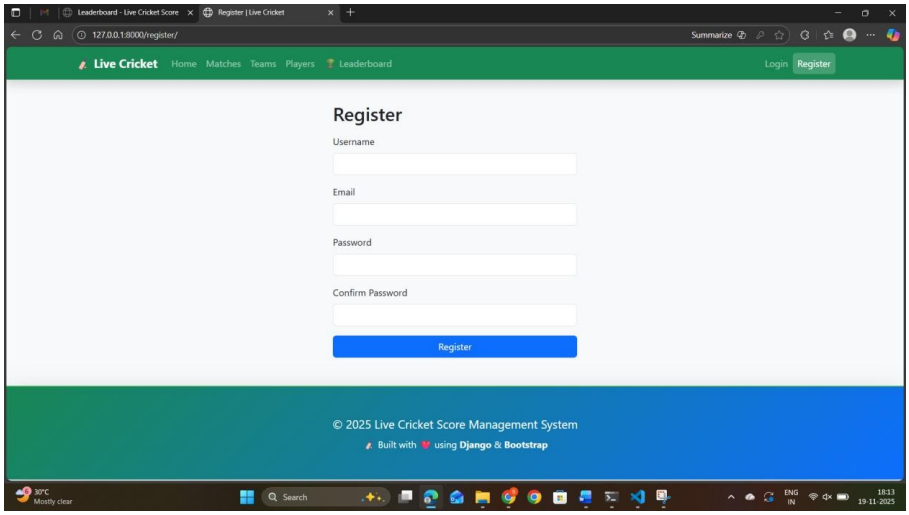


Figure 6.2: Registration Page

6.3 Login Page

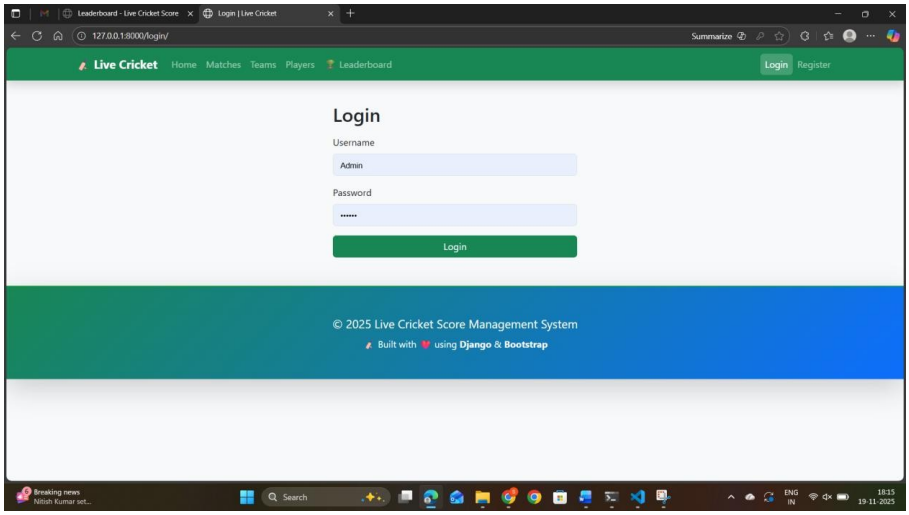


Figure 6.3: Login Page

6.4 Login Successful Page

After a successful registration or login, a welcome message is displayed and the user name is shown in the navigation bar.

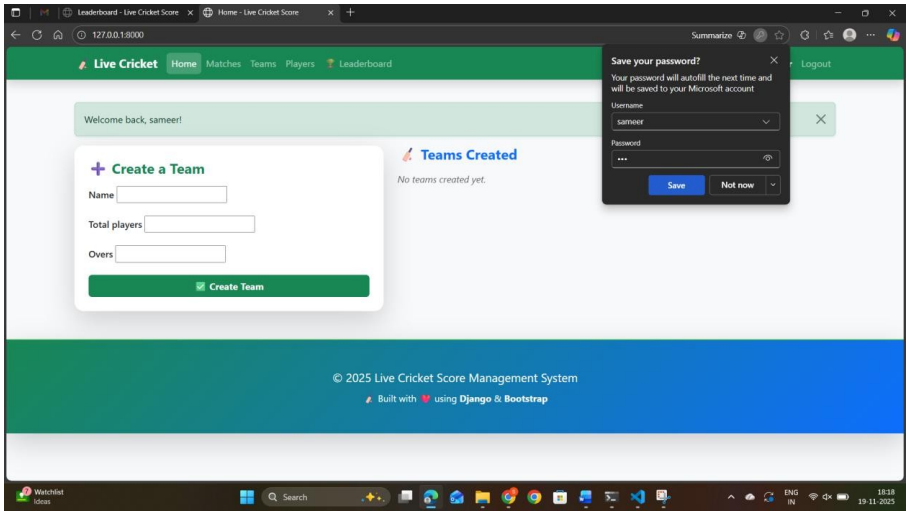


Figure 6.4: Registration / Login Successful Page

6.5 Matches Page

The Matches page allows a new match to be scheduled and displays each match with scores, winner and a Live Score Control button.

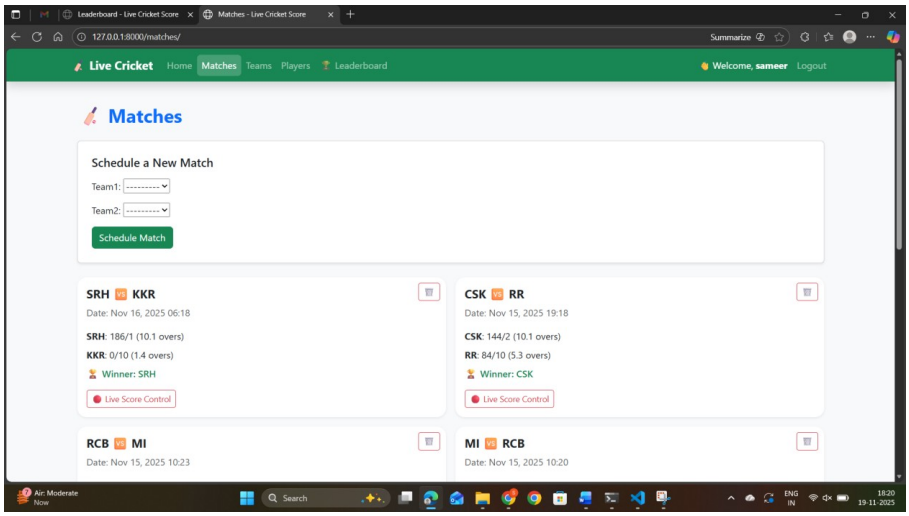


Figure 6.5: Matches Page

6.6 Team Page

The Team Management and Rosters page shows each team with its players, statistics (played, won, lost, points) and Edit, Delete and Add Player buttons.

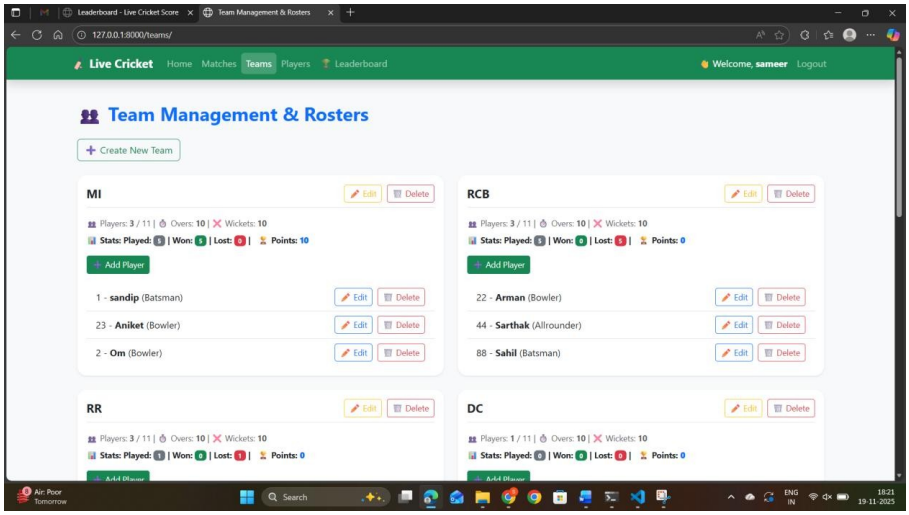


Figure 6.6: Team Page

6.7 Players Page

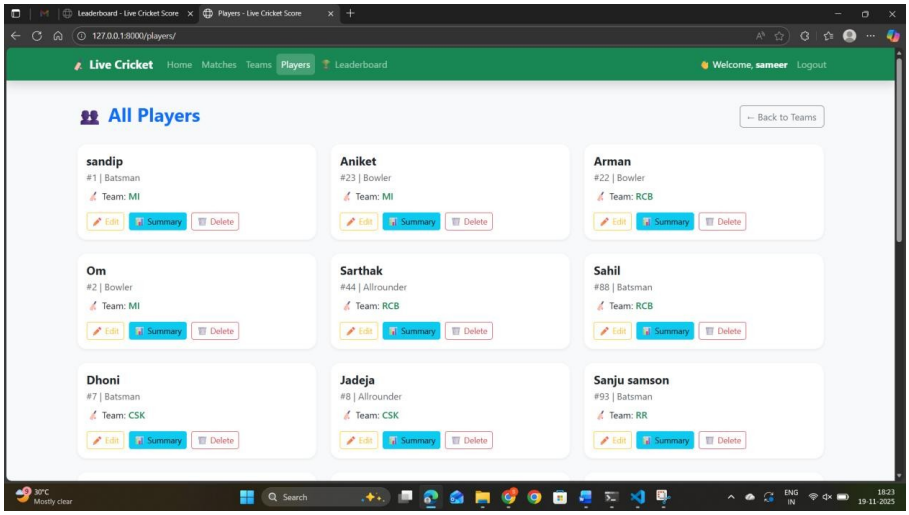


Figure 6.7: Players Page

6.8 Live Scoring Panel

The live scoring panel shows the batting team score, overs, target, striker, non-striker, bowler and the controls to record runs, extras, wicket and End Innings.

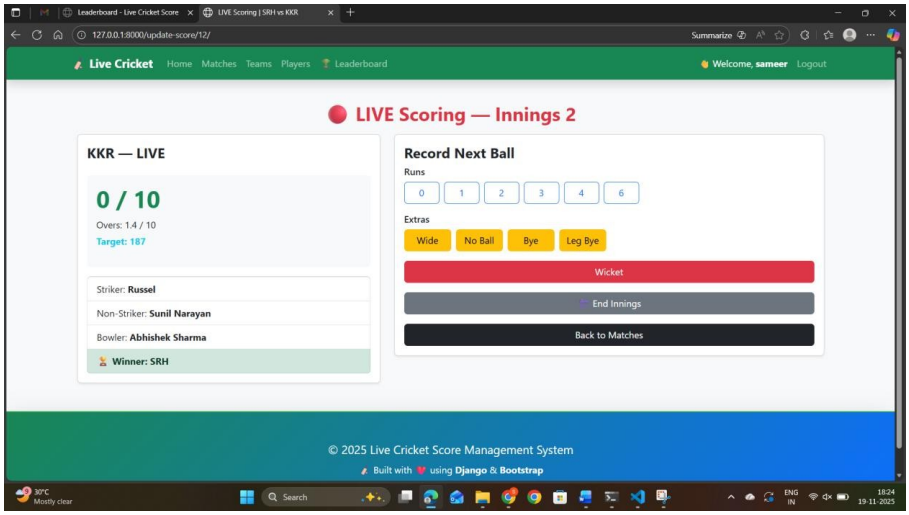


Figure 6.8: Live Scoring Panel

6.9 Leaderboard Page

The Team Leaderboard ranks teams by points (win = 2 points, loss = 0 points).

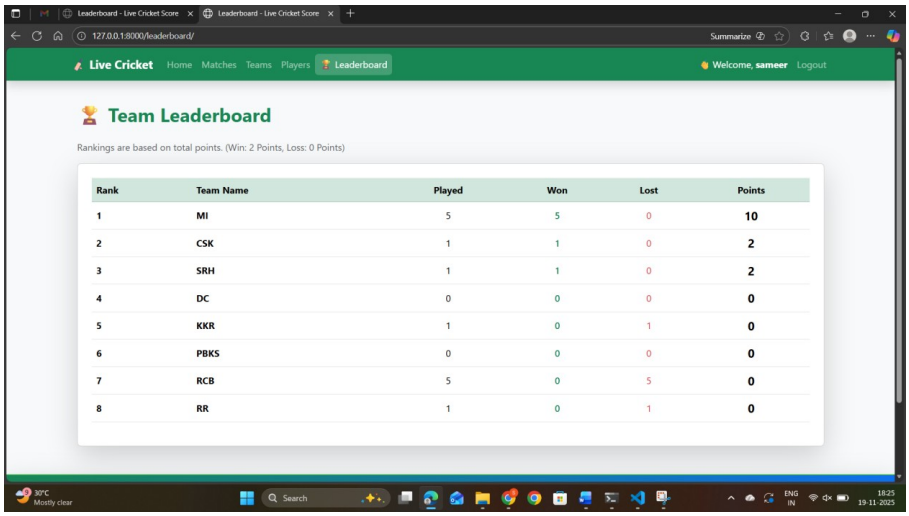


Figure 6.9: Leaderboard Page

6.10 Admin Page

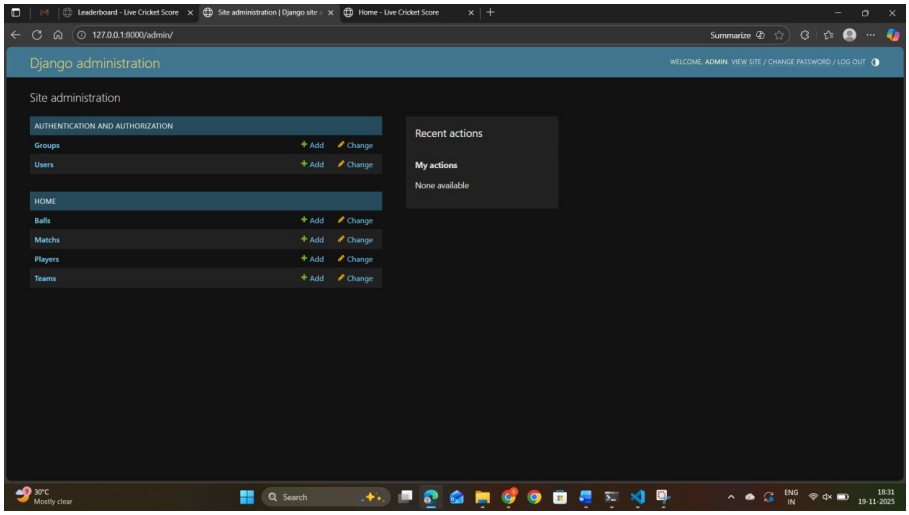


Figure 6.10: Django Admin Page

7. CONCLUSIONS

7.1 Conclusion

Live Cricket Score Management System successfully achieved its objective of providing a simple, efficient and accurate digital platform for cricket scoring. The system was developed using Django, SQLite and Bootstrap, and it provides team and player management, match setup, live ball-by-ball scoring, innings handling, automatic winner decision and a points-based leaderboard.

The key phases of requirement analysis, system design, implementation and testing were carried out systematically following the Waterfall Model. The project gave practical experience in full-stack web development, database design with the Django ORM, user-interface design, modular programming, data validation and software testing.

Overall, the project demonstrates that a digital scoring platform can significantly improve the efficiency, accuracy and transparency of cricket scoring through automation and organised record keeping.

7.1.1 Significance of the System

- Replaces error-prone manual scorebooks with automatic calculation.
- Stores every ball, so complete match history is available for review.
- Maintains the team leaderboard automatically after every match.
- Provides a responsive interface usable on mobile, tablet and desktop.
- Uses free, open-source technologies, which makes it suitable for schools, colleges and local clubs.

7.1.2 Limitations of the System

- SQLite and a single Django server are suitable only for small and medium usage.
- No role-based access control beyond logged-in users and Django Admin.
- No live sharing of the scoreboard to remote viewers, push notifications or commentary.
- Winner decision handles only the basic result; tie-breaks, super overs and DLS are not supported.
- No advanced player statistics such as strike rate graphs, economy charts or career records.
- Development settings (DEBUG = True) must be changed before production deployment.
- No automated large-scale performance testing has been performed.

7.1.3 Future Scope

1. **Advanced analytics:** strike rate graphs, bowler economy charts, wagon wheels and performance history.

2. **Tournament management:** fixtures, scheduling, knockout rounds and automated ranking.
3. **Live commentary and streaming integration** for better engagement.
4. **Mobile applications** for Android and iOS with offline scoring.
5. **Multi-language support** such as Marathi and Hindi.
6. **Cloud-based real-time sync** using Firebase or AWS for multiple devices and viewers.
7. **AI-based score and win-probability prediction.**
8. **Enhanced security:** role-based access (Admin / Umpire / Scorer), two-factor authentication and audit logs.
9. **Offline mode scoring** with automatic synchronisation.
10. **Integration with digital LED / TV scoreboards** at grounds.

REFERENCES

1. Cricket Rules and Existing Platforms

- International Cricket Council (ICC) - Playing Conditions and Rules. https://www.icc-cricket.com/
- Marylebone Cricket Club (MCC) - Laws of Cricket.
- CricHeroes, CricClubs and ESPNcricinfo - existing digital scoring platforms studied for features.

2. Technical and Technology Documentation

- Django Documentation (2024). https://docs.djangoproject.com/
- Bootstrap Official Documentation (2024). https://getbootstrap.com/
- MDN Web Docs - HTML, CSS and JavaScript (2024). https://developer.mozilla.org/
- Python Software Foundation (2024). Python Documentation. https://www.python.org/doc/
- SQLite Documentation (2024). https://www.sqlite.org/docs.html
- W3Schools Web Tutorials (2024). https://www.w3schools.com/

3. Software Engineering Books

- Pressman, R. S., & Maxim, B. R. (2014). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill Education.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
- Fowler, M. (2003). UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd ed.). Addison-Wesley.
- Bass, L., Clements, P., & Kazman, R. (2012). Software Architecture in Practice (3rd ed.). Addison-Wesley.

4. Development Tools

- Visual Studio Code and PyCharm - code development.
- Git and GitHub - version control and repository management.
- Google Chrome / Mozilla Firefox - application testing.

APPENDIX A: USER MANUAL

This appendix explains how the Admin / Scorer and the Viewer use the Live Cricket Score Management System. The screens referred to are shown in Chapter 6 (Results).

A.1 Starting the Application

1. Install Python 3.x and Django 5.2.6, then open the project folder in Visual Studio Code or a terminal.
2. Apply the database migrations with **python manage.py migrate**.
3. Create the Admin account with **python manage.py createsuperuser** (optional, needed for the Django Admin panel).
4. Start the server with **python manage.py runserver** and open **http://127.0.0.1:8000/** in Chrome, Firefox or Edge.

A.2 Admin / Scorer Workflow

1. **Register and log in.** Open the Register page, enter a username, email and password, and submit. After a successful login a welcome message and the user name appear in the navigation bar.
2. **Create teams.** On the Home page enter the team name, total players (for example 11) and overs per innings. The wicket limit is calculated automatically as total players minus one.
3. **Add players.** On the Teams page press Add Player under a team and enter the jersey number, name and role (Batsman, Bowler or All-rounder). Players can be edited or deleted later.
4. **Schedule a match.** On the Matches page select Team 1 and Team 2 and press Schedule Match. The match card appears with its scores, winner and a Live Score Control button.
5. **Select opening players.** Press Live Score Control and choose the striker, non-striker and bowler in the pop-ups.
6. **Record every ball.** Press a run button (0, 1, 2, 3, 4, 6), an extra button (Wide, No Ball, Bye, Leg Bye) or Wicket. After a wicket, select the next batsman; after every over, select the next bowler.
7. **End the first innings.** When the team is all out or the overs are completed, press End Innings. The second innings starts and the target is displayed on the scoring panel.
8. **Finish the match.** When the target is reached, the team is all out or the overs are completed, the winner is decided and the leaderboard is updated automatically.

A.3 Viewer Workflow

- Open the Matches page to see each match with scores, overs and the winner.
- Open the Teams page to see the roster, matches played, won, lost and points of every team.
- Open the Players page and press Summary on a player card to see the player summary.
- Open the Leaderboard page to see teams ranked by points (win = 2 points, loss = 0 points).

A.4 Common Problems and Solutions

Problem	Likely Cause	Solution
Scoring or team pages ask for login	Session expired or user not logged in	Log in again; scoring operations need an authenticated user.
Team cannot be created	Blank or duplicate team name	Enter a unique team name; the name field is unique.
Wicket limit not as expected	Total players entered incorrectly	Edit the team; the wicket limit is recalculated as total players minus one.
Over count did not increase	The ball was a Wide or No Ball	This is correct: wides and no-balls are illegal deliveries and do not count in the over.
Leaderboard did not change	Match not completed, or winner already decided	Finish the second innings; results are counted only once per match.
Page looks unstyled	No internet for CDN Bootstrap files	Connect to the internet; Bootstrap is loaded from a CDN.

APPENDIX B: WORKED EXAMPLE OF SCORING

This example shows how the scoring rules of Section 4.3.3 act on a single over. Batsmen A and B are at the crease (A on strike) and the bowler is X. The example is illustrative and follows the rules implemented in the system: wides and no-balls are illegal deliveries, odd runs and the end of an over change the strike, and a wicket brings in a new batsman.

Table B.1: Scoring of One Over

Delivery	Scorer Input	Ball Record	Legal?	Total	On Strike After
0.1	1 run	runs = 1	Yes	1/0	B (odd run)
0.2	4 runs	runs = 4	Yes	5/0	B
Wide	Wide	extra_type = wide, runs = 1	No	6/0	B
0.3	0 runs	runs = 0	Yes	6/0	B
0.4	Wicket (B out)	is_wicket = True, player_out = B	Yes	6/1	C (new batsman)
0.5	1 run	runs = 1	Yes	7/1	A (odd run)
0.6	2 runs	runs = 2	Yes	9/1	C (end of over)

Result of the over: 9 runs and 1 wicket. Seven Ball records are stored for the over, but only six are legal deliveries, so the over completes after the sixth legal ball. At that point the scorer is asked to select the next bowler and the strike is swapped.

B.1 Example of Winner Decision

Team Alpha bats first in a 10-over match and scores 120/6. Pressing End Innings starts the second innings with a target of 121. Team Beta finishes its 10 overs on 118/8. Because team2_runs is lower than team1_runs after the second innings, decide_winner() declares Team Alpha the winner and updates the Team table once:

Team	Played	Won	Lost	Points
Team Alpha	+1	+1	0	+2
Team Beta	+1	0	+1	0

If the winner is already stored, a second call to decide_winner() returns immediately, so the statistics are never counted twice. If both teams score the same runs, no winner is set, both teams receive a match played and no points are added (a tie), since super overs are outside the present scope.

B.2 Wicket Limit Example

A team created with 11 players gets a wicket limit of 10. The innings therefore ends automatically at 10 wickets or when the allotted overs are completed, whichever comes first, and the second innings can then begin.

APPENDIX C: URL ROUTES AND SOURCE MAPPING

The table below lists the URL routes defined in home/urls.py, the view that handles each route and the purpose of the page or action. It helps in understanding how the Model-View-Template pattern connects the browser requests to the stored data.

Table C.1: Application Routes

URL Pattern	View Function	Purpose
login/	login_view	Authenticate the user
register/	register_view	Create a new user account
logout/	logout_view	End the user session
(home)	home	Create a team and list teams
matches/	matches	Schedule matches and list results
teams/	teams	Team cards, rosters and statistics
players/	players	List of all players
leaderboard/	leaderboard_view	Teams ranked by points
add-player/<team_id>/	add_player	Add a player to a team
edit-team/ and delete-team/	edit_team, delete_team	Edit or delete a team
edit-player/ and delete-player/	edit_player, delete_player	Edit or delete a player
match-setup/<match_id>/	match_setup	Select striker, non-striker, bowler
match/<id>/live/ and update-score/<id>/	update_score	Live ball-by-ball scoring panel
player/<player_id>/summary/	player_summary	Player summary page
delete-match/<match_id>/	delete_match	Remove a match

C.1 Request Flow

A request from the browser is matched by cricket/urls.py, which hands over to home/urls.py. The selected view reads or updates the Team, Player, Match and Ball models through the Django ORM, and the result is rendered with a template that extends templates/base.html. Business rules such as the wicket limit, strike swap and winner decision are kept inside the model methods, so the views remain short and the rules can be reused and tested independently.

APPENDIX D: REQUIREMENT TRACEABILITY AND COMPARISON

D.1 Requirement Traceability Matrix

Table D.1 links the functional requirements of Table 3.1 to the module that implements them and the test cases of Tables 4.8 and 5.2 that verify them.

Table D.1: Requirement Traceability

Requirement	Module	Test Cases
FR-01 Register, login, logout	Authentication and Access	T01, T02, T03, T23
FR-02, FR-03 Teams and wicket limit	Team Management	T04, T05, T06, TC01, TC02
FR-04 Players	Player Management	T07
FR-05 Schedule match	Match Setup	T08, T09, TC03
FR-06 Select striker, non-striker, bowler	Live Scoring	T14, T15
FR-07, FR-08 Runs, extras, wickets	Live Scoring	T10-T14, TC04-TC06
FR-09, FR-10 Strike rotation and display	Live Scoring	T15, T16, TC07
FR-11, FR-12 Innings, target, End Innings	Innings Handling	T17, T18
FR-13 Winner and statistics	Scoreboard and Results	T19, T20, TC08, TC11
FR-14, FR-15 Leaderboard and players	Leaderboard	T21, TC09, TC12
FR-16, FR-17, FR-18 Storage, delete, Admin	Admin Data Management	T22, TC09, TC12

D.2 Comparison with Existing Approaches

Table D.2 compares the proposed system with paper scorebooks and with large commercial platforms in general terms.

Table D.2: Comparison

Feature	Paper Scorebook	Large Commercial Apps	This System
Automatic totals and overs	No	Yes	Yes
Ball-by-ball history stored	Manual	Yes	Yes (Ball model)
Customisable for local needs	Yes	Limited	Yes (open source)
Cost	Low	Often paid features	Free tools
Team leaderboard	Manual	Yes	Yes (2 points per win)
Advanced analytics, live streaming	No	Yes	Not in present version

APPENDIX E: DEPLOYMENT CHECKLIST

The project runs with development settings for demonstration. Section 4.5 notes that these must be changed before a public deployment. The following checklist summarises the required steps.

1. Set **DEBUG = False** in cricket/settings.py.
2. Replace the development **SECRET_KEY** with a long random value stored outside the source code.
3. Configure **ALLOWED_HOSTS** with the domain name or IP address of the server.
4. Run **python manage.py collectstatic** and serve static files through the web server.
5. Use a production WSGI server (for example Gunicorn or uWSGI) behind a web server such as Nginx instead of runserver.
6. Enable HTTPS and take regular backups of the database file (db.sqlite3).
7. For many simultaneous users, change the DATABASES setting from SQLite to MySQL; the Django ORM needs no other code change.

APPENDIX F: GLOSSARY

Term	Meaning
Over	A set of six legal deliveries bowled by one bowler.
Striker / Non-striker	The batsman facing the ball and the batsman at the other end.
Wide / No Ball	Illegal deliveries; a run is added and the ball is not counted in the over.
Bye / Leg Bye	Runs taken without the ball touching the bat; the ball is counted as a legal delivery.
Innings	The period in which one team bats; each match has two innings.
Target	Runs required to win in the second innings (first innings total plus one).
Wicket limit	Maximum wickets in an innings, calculated as total players minus one.
MVT	Model-View-Template, the architecture pattern used by Django.
ORM	Object-Relational Mapper that maps Django models to database tables.
CSRF	Cross-Site Request Forgery, an attack prevented by Django CSRF tokens.
CDN	Content Delivery Network used here to load Bootstrap files.